

Attribution-Patching-Guided Replay Refinement for Task-Vector Merging

Anonymous Authors

Draft proposal

July 9, 2026

Abstract

We study the task of merging fine-tuned variants of a shared pretrained model when a small labeled replay buffer for each task is available. After an initial task-arithmetic merge, we refine the merged model with task gradients while using the known task experts as coordinate-wise trust anchors. The resulting update admits two equivalent motivations that we develop in parallel. As a parameter-space analogue of attribution patching, the effect of moving the current model toward a task expert is approximated by a first-order loss attribution, and only coordinates for which the expert direction is locally loss-decreasing are used. As an expert-anchored trust-region replay refinement, the departure from ordinary replay gradient descent is that each coordinate’s step is weighted by its distance to the expert, gated to keep only steps that move toward the expert, and clipped so it cannot overshoot—the same coordinates that the attribution view selects. Both perspectives lead to a single refinement algorithm.

1 Related Work and Background

Model merging and weight-space averaging. Model merging combines multiple trained or fine-tuned models into a single model in weight space, avoiding the inference-time overhead of ensembling. Early and influential forms include direct parameter averaging, model soups, and Fisher-weighted merging. Model soups average multiple fine-tuned checkpoints and can improve accuracy without increasing inference cost when the checkpoints lie in a compatible basin (Wortsman et al., 2022). Fisher merging instead weights parameters using Fisher information, motivated by a Laplace approximation to the local posterior around each model (Matena and Raffel, 2022). These approaches motivate the broader view that homologous fine-tuned models can often be combined directly in parameter space, but they do not by themselves resolve interference when the merged models specialize in different downstream tasks.

Task vectors. Task arithmetic represents the effect of fine-tuning as a task vector

$$\tau_t = \theta_t - \theta_0, \tag{1}$$

where θ_0 is a shared pretrained model and θ_t is the model fine-tuned on task t . A merged model is then constructed as

$$\theta_{\text{merge}} = \theta_0 + \sum_{t \in \mathcal{T}} \lambda_t \tau_t, \tag{2}$$

where λ_t are scalar or tensor-wise coefficients. Task Arithmetic demonstrated that such vectors can be added or negated to steer model behavior (Ilharco et al., 2022). This formulation is attractive

because it requires only checkpoints, but naive addition can over-count redundant directions and introduce sign or magnitude conflicts across tasks.

Interference-aware task-vector merging. A major line of work addresses interference between task vectors. TIES-Merging trims small-magnitude updates, resolves sign disagreement, and merges only sign-consistent components (Yadav et al., 2023). DARE randomly drops most delta parameters and rescales the remaining entries, arguing that fine-tuning deltas are highly redundant (Yu et al., 2023). DELLA generalizes this drop-and-rescale idea with magnitude-based sampling (Deep et al., 2024). Model Breadcrumbs similarly builds sparse masks by removing both negligible and outlier perturbations (Davari and Belilovsky, 2023). Localize-and-Stitch explicitly localizes sparse task-relevant regions and stitches only those regions back into the pretrained model (He et al., 2024). These methods share the view that not all task-vector coordinates should be merged equally. The present proposal does not introduce another pruning rule; instead, it asks whether a small amount of supervised replay can repair an initial merge while staying close to the known task experts.

Data-dependent merging baselines. Several merging methods use small amounts of data or model outputs to estimate better combinations. Activated Parameter Locating (APL) estimates task-vector importance through a data-dependent contrast and then uses the result to guide pruning (Kong et al., 2024). RegMean formulates merging as a regression problem that minimizes prediction differences between the merged model and the candidate models (Jin et al., 2023). AdaMerging learns task-wise or layer-wise merge coefficients by minimizing an unsupervised entropy objective on unlabeled samples (Yang et al., 2024). These methods are important baselines because the proposed method also uses data, although it uses labeled replay buffers only for a few post-merge refinement steps rather than for constructing a pre-merge mask or a closed-form fusion rule.

Attribution patching. Attribution patching approximates the effect of an intervention by a first-order Taylor expansion, replacing many explicit interventions by gradient-based scores (Syed et al., 2024; Kramar et al., 2024). In parameter space, the analogous approximation is a directional derivative of the task loss along a chosen parameter displacement. Here the relevant intervention is not a replacement of one expert by another. It is the movement from the current merged model toward the expert for task i . This gives a direct bridge between attribution patching and replay-buffer refinement: the attribution score decides which coordinates of the expert direction are locally predicted to decrease the task loss.

Shared and task-specific structure. Several recent methods recognize that task vectors contain both shared and exclusive information. Twin-Merging separates shared and exclusive task knowledge and dynamically combines them at inference time (Lu et al., 2024). Task Vector Bases clusters similar task vectors to compress and reuse shared directions, providing a theoretical and practical framework for reducing redundancy across related tasks (Zeng et al., 2025). FusionBench documents the need for standardized evaluation across model-fusion methods and includes GLUE-style language tasks among its benchmark families (Tang et al., 2024). Our proposal is complementary: rather than interpreting small attribution values as evidence that a coordinate can be pruned, we use the sign of a merge-to-expert attribution only as a local gate for refinement.

Benchmark settings. We use the standard GLUE-8 merging setting as the primary diagnostic benchmark: CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, and RTE, excluding WNLI as is common in many GLUE model-merging studies. GLUE was introduced as a multi-task benchmark

for natural language understanding (Wang et al., 2018). We use RoBERTa-base as the primary encoder backbone because it is a strong, widely used model for GLUE-style evaluation (Liu et al., 2019). In a full fine-tuning setup, each task has a task-specific classifier or regression head. We therefore treat RoBERTa GLUE-8 as a diagnostic setting for representation merging: the shared encoder is merged and evaluated with the corresponding task-specific head. This multi-head setup is convenient and common, but it assumes oracle task identity at evaluation time and should not be the only evidence for the method. Later experiments should include a shared-output setting, for example GLUE or SuperGLUE in a text-to-text T5-style format (Wang et al., 2019; Raffel et al., 2020). A second-modality benchmark is also important: the standard CLIP/ViT task-vector suite with image-classification tasks such as SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, and DTD gives a stronger test of whether the method is specific to GLUE encoders or to task-specific NLP heads (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024).

2 Methodology

2.1 Problem Setup

Let $\mathcal{T} = \{1, \dots, T\}$ denote a set of tasks. All task experts are initialized from the same pretrained encoder $\theta_0 \in \mathbb{R}^d$. Fine-tuning on task t gives expert encoder parameters θ_t and task vector

$$\tau_t = \theta_t - \theta_0. \quad (3)$$

The mergeable encoder parameters are treated as a vector in $\mathbb{R}^d$. We write $r \in \{1, \dots, d\}$ for a parameter coordinate. Task-specific heads are not merged in the primary GLUE experiments; the merged encoder is evaluated with the corresponding head for each task.

The initial merged encoder is

$$\theta^{(0)} = \theta_0 + \sum_{i=1}^T \lambda_i \tau_i, \quad (4)$$

where λ_i are fixed task-arithmetic coefficients, typically uniform or chosen by a small grid search. Let $\mathcal{D}_i^{\text{probe}}$ be a small replay buffer used for refinement, and let $\mathcal{D}_i^{\text{eval}}$ be the evaluation split used only for reporting performance. For each task i , let

$$\mathcal{L}_i(\theta; \mathcal{D}_i^{\text{probe}}) \quad (5)$$

denote the replay-buffer loss obtained by using encoder parameters θ with task i 's prediction head. The proposed method does not use a base-relative saliency term and does not construct a pre-merge pruning mask. Its only data-dependent operation is a short replay-buffer refinement of the initial merge.

2.2 Trust-Region Replay Refinement

Starting from the task-arithmetic initialization in Equation (4), the natural data-dependent baseline is ordinary replay-buffer gradient descent, which takes an unconstrained step along the task gradients:

$$\theta^{(s+1)} = \theta^{(s)} - \eta \sum_{i=1}^T \nabla_{\theta} \mathcal{L}_i(\theta^{(s)}; \mathcal{D}_i^{\text{probe}}). \quad (6)$$

This baseline treats the merge like any other initialization and lets the replay gradients move each coordinate freely, with nothing tying the refined model to the known task experts. The central

departure of our method is precisely this anchoring: rather than taking a free gradient step, every coordinate of the update is tied to the corresponding task expert. For each task i and coordinate r we (i) weight the negative-gradient step by the current distance $|v_{i,r}|$ to the expert, so that coordinates already close to the expert barely move while coordinates far from it move more; (ii) keep the step only where it moves the model *toward* the expert, discarding coordinates whose replay gradient would pull away from it; and (iii) clip the resulting step to the expert-distance trust region so that a single update can never overshoot the expert. These three operations—expert-distance weighting, the expert-agreement gate, and the trust-region clip—are what distinguish the refinement from ordinary replay gradient descent, and together they keep the whole refinement anchored to the known experts rather than letting it drift.

A secondary and comparatively minor design choice concerns how the per-task updates are scheduled within a refinement step. An aggregated expert-guided update would compute all u_i at $\theta^{(s)}$ and then apply $\eta \sum_i u_i$ at once; we instead apply each task’s update immediately after computing it, so that its gradient, expert direction, and gate are evaluated at the same model state to which it is applied. Let $\theta^{(s,0)} = \theta^{(s)}$, and let π_s be the task order for sweep s ; π_s can be fixed, cyclically rotated, or randomly permuted. For within-sweep index $k \in \{1, \dots, T\}$, set $i = \pi_s(k)$ and compute

$$g_i^{(s,k)} = \nabla_{\theta} \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}}), \quad (7)$$

$$v_i^{(s,k)} = \theta_i - \theta^{(s,k-1)}, \quad (8)$$

$$m_{i,r}^{(s,k)} = \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}, \quad (9)$$

$$\tilde{u}_{i,r}^{(s,k)} = -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}. \quad (10)$$

From this optimization side the coordinate mask $m_{i,r}^{(s,k)}$ arises as a trust-region acceptance rule rather than as an attribution score: we keep only those coordinates whose replay-gradient step also moves the current model toward expert i , and discard the coordinates where the replay step would pull away from the expert and out of the expert-anchored trust region. This is the same condition $g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0$ that Section 2.3 obtained from the attribution-patching sign in Equation (19), now read as *stay within the expert-anchored trust region* instead of *move along a locally loss-decreasing intervention*. The two perspectives therefore agree coordinate by coordinate. The raw update is first scaled by the learning rate η and *then* clipped by the the current distance to the expert, so that the applied per-coordinate step is bounded by $|v_{i,r}^{(s,k)}|$ regardless of η ,

$$u_{i,r}^{(s,k)} = \text{clip}\left(\eta \tilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|\right), \quad (11)$$

and the current model is immediately updated with this clipped step:

$$\theta^{(s,k)} = \theta^{(s,k-1)} + u_i^{(s,k)}. \quad (12)$$

After all tasks in the sweep have been processed, we set

$$\theta^{(s+1)} = \theta^{(s,T)}. \quad (13)$$

The robustness of the refinement comes primarily from the expert anchoring rather than from this scheduling: the distance weighting and gate keep every coordinate pointed at its expert, and the trust-region clip caps how far it can travel. The immediate application of each update, though secondary to the anchoring, has two motivations of its own. First, each update is applied at the

same local model state on which its gradient, expert direction, and gate were computed. Second, the trust-region clipping in Equation (11) controls each single-task movement before the next task sees the modified model, rather than allowing several task vectors to accumulate into one larger coordinate step. Because the clip is applied *after* the learning-rate scaling, the per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η ; for ≤ 1 a single update can never overshoot the expert in a coordinate, so the refinement stays within the expert-distance trust region and cannot diverge no matter how large η is. The learning rate then controls only how many coordinates are driven to the trust-region boundary, not the maximum per-coordinate step size. The sequential rule still does not guarantee Pareto improvement across tasks, because an update that helps task i can hurt task j . We therefore treat task ordering, cross-task interference, and worst-task retention as explicit diagnostics rather than as consequences of the gate alone. In practice, one can also normalize $u_i^{(s,k)}$ by tensor-wise RMS before applying the single-task update; the defining operation remains the coordinate gate in Equation (19), whether it is read as an attribution-patching sign or as a trust-region acceptance rule. Under either reading the procedure is the single algorithm summarized in Algorithm 1.

Algorithm 1 Sequential expert-direction-gated replay refinement

Require: Pretrained encoder θ_0 ; task experts $\{\theta_i\}_{i=1}^T$; task heads; replay buffers $\{\mathcal{D}_i^{\text{probe}}\}_{i=1}^T$; task-arithmetic weights $\{\lambda_i\}_{i=1}^T$; steps S ; learning rate η ; gate threshold ϵ_{gate} .

- 1: $\tau_i \leftarrow \theta_i - \theta_0$ for all tasks i .
- 2: $\theta^{(0)} \leftarrow \theta_0 + \sum_{i=1}^T \lambda_i \tau_i$.
- 3: **for** $s = 0, \dots, S - 1$ **do**
- 4: $\theta^{(s,0)} \leftarrow \theta^{(s)}$.
- 5: Choose a task order π_s over $\{1, \dots, T\}$.
- 6: **for** $k = 1, \dots, T$ **do**
- 7: $i \leftarrow \pi_s(k)$.
- 8: $g_i^{(s,k)} \leftarrow \nabla_{\theta} \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}})$ using task i 's head.
- 9: $v_i^{(s,k)} \leftarrow \theta_i - \theta^{(s,k-1)}$.
- 10: **for** each mergeable coordinate r **do**
- 11: $m_{i,r}^{(s,k)} \leftarrow \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}$.
- 12: $\tilde{u}_{i,r}^{(s,k)} \leftarrow -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}$.
- 13: $u_{i,r}^{(s,k)} \leftarrow \text{clip}(\eta \tilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|)$. ▷ clip after learning-rate scaling
- 14: **end for**
- 15: $\theta^{(s,k)} \leftarrow \theta^{(s,k-1)} + u_i^{(s,k)}$.
- 16: **end for**
- 17: $\theta^{(s+1)} \leftarrow \theta^{(s,T)}$.
- 18: **end for**
- 19: **return** $\theta^{(S)}$.

2.3 A Second Perspective: Attribution Patching in Parameter Space

The refinement procedure developed in the previous section (Algorithm 1) can be motivated from a second independent starting point that arrive at the same per-coordinate update, comin from an interpretability-style argument: it treats each move toward a task expert as a parameter-space intervention and uses a first-order attribution-patching score to decide, coordinate by coordinate, whether that intervention is locally loss-decreasing.

The attribution step is a local calculation around the current model state. To make the sequential update rule explicit, let ϑ denote the current merged encoder immediately before applying a single task update. For task i , the task expert θ_i is a known reference solution, and the merge-to-expert direction is

$$v_i(\vartheta) = \theta_i - \vartheta. \quad (14)$$

The corresponding parameter-space intervention is the path

$$\Gamma_i(\alpha; \vartheta) = \vartheta + \alpha v_i(\vartheta), \quad \alpha \in [0, 1], \quad (15)$$

where $\alpha = 0$ is the current model and $\alpha = 1$ is the task expert. Let

$$g_i(\vartheta) = \nabla_{\theta} \mathcal{L}_i(\vartheta; \mathcal{D}_i^{\text{probe}}) \quad (16)$$

be the task gradient at the current model, evaluated with task i 's head. A first-order attribution-patching approximation to the loss change caused by moving from ϑ toward expert i is

$$\Delta_i^{\text{AP}}(\vartheta) = \langle g_i(\vartheta), v_i(\vartheta) \rangle = \sum_{r=1}^d a_{i,r}^{\text{AP}}(\vartheta), \quad (17)$$

with coordinate-wise contribution

$$a_{i,r}^{\text{AP}}(\vartheta) = g_{i,r}(\vartheta) v_{i,r}(\vartheta). \quad (18)$$

A negative contribution means that moving coordinate r toward the task expert is locally predicted to reduce task i 's loss. A positive contribution means that the same movement is locally predicted to increase the loss. Thus the loss-decreasing expert gate is

$$m_{i,r}(\vartheta) = \mathbf{1}\{a_{i,r}^{\text{AP}}(\vartheta) < -\epsilon_{\text{gate}}\}, \quad (19)$$

where $\epsilon_{\text{gate}} \geq 0$ is an optional confidence threshold; the default method uses $\epsilon_{\text{gate}} = 0$. The condition $g_{i,r}(\vartheta) v_{i,r}(\vartheta) < 0$ is exactly the condition that the expert direction agrees with the negative gradient in coordinate r .

The resulting coordinate-wise update for task i is a distance-scaled negative-gradient step, masked by the attribution-patching sign:

$$\tilde{u}_{i,r}(\vartheta) = -g_{i,r}(\vartheta) |v_{i,r}(\vartheta)| m_{i,r}(\vartheta). \quad (20)$$

Whenever $m_{i,r}(\vartheta) = 1$, this update points toward the expert because $-g_{i,r}(\vartheta)$ and $v_{i,r}(\vartheta)$ have the same sign. The factor $|v_{i,r}(\vartheta)|$ makes the step small when the current model is already close to the expert in that coordinate and larger when the coordinate is far from the expert. Importantly, the gate only certifies a local first-order effect for task i at the current model state. It does not by itself prove that the coordinate is harmless for other tasks, so cross-task interference must be measured experimentally.

3 Experimental Protocol

Primary diagnostic setting: RoBERTa GLUE-8. The primary benchmark is GLUE-8: CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, and RTE. We use RoBERTa-base as the default encoder. Each task expert is obtained by fine-tuning from the same pretrained checkpoint with matched hyperparameter budgets. The primary setting merges full fine-tuned encoders and keeps task-specific heads fixed for evaluation. This setting is intentionally described as *oracle-task-id, multi-head encoder merging*; it is a useful and inexpensive diagnostic but not a complete demonstration of single-model deployment.

Recommended proof-of-concept order. The first experiments should be deliberately smaller than full GLUE-8, but the final report should avoid relying only on hand-picked related tasks.

1. **Code smoke test:** use a distilled RoBERTa-style encoder (Sanh et al., 2019) on MRPC and STS-B. This pair is small enough for rapid iteration and tests whether the implementation can exploit related sentence-pair tasks.
2. **First convincing three-task result:** use RoBERTa-base on MRPC, STS-B, and SST-2. MRPC and STS-B form a related sentence-pair pair, while SST-2 acts as a less-related single-sentence control task.
3. **Systematic pairwise GLUE study:** evaluate all pairwise merges among the eight GLUE tasks and report a heatmap of mean retention, worst-task retention, and expert-relative drop. This guards against cherry-picking a favorable task subset.
4. **Structured GLUE subsets:** evaluate related subsets (MRPC/STS-B and MRPC/QQP/STS-B), a mixed-format subset (STS-B/SST-2/QNLI), small-task stress subsets (RTE/CoLA/MRPC and QQP/RTE/CoLA), and the entailment cluster (MNLI/QNLI/RTE).
5. **Full GLUE-8 scale-up:** run the final method and all feasible baselines on all eight tasks. CoLA and RTE require multiple seeds because they are small or high-variance; MNLI requires explicit compute budgeting because it is much larger than the other tasks.

The main paper-aligned result should use RoBERTa-base. Distilled models are useful for debugging hyperparameters and implementation errors but should not be the only evidence.

Secondary benchmark settings. To address the limitations of multi-head GLUE, we include two secondary benchmark tracks once the RoBERTa proof of concept is stable. First, a shared-output NLP track evaluates GLUE or SuperGLUE in a text-to-text format using T5-style models, so that the merged model uses a common generation head rather than task-specific classifiers (Wang et al., 2019; Raffel et al., 2020). Second, a CLIP/ViT vision track evaluates the standard task-vector image-classification suite: SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, and DTD (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024). The vision track is especially valuable because it tests a different modality, different loss geometry, and stronger domain shifts than GLUE. A small LoRA- or adapter-based LLM merging experiment can be added later if compute permits, but it should not replace the controlled RoBERTa and CLIP/ViT studies.

Replay data. For refinement, we use $n \in \{4, 8, 16, 32, 64, 128\}$ replay examples per task. Replay examples are sampled from training data or from a held-out split separated from the final validation split. For classification tasks, replay buffers should be class-balanced whenever possible; for imbalanced tasks such as QQP and for small tasks such as RTE, we report the exact sampling procedure. We report sensitivity to the number of replay examples, variation across replay samples, and the area under the replay-budget curve rather than only the best tuned replay size.

Baselines. Because the proposed method uses labeled replay buffers, baselines are grouped by data regime.

Checkpoint-only baselines.

Pretrained encoder, individual single-task experts, uniform model averaging, task arithmetic, Fisher merging, TIES, DARE, DELLA-style magnitude sampling, Model Breadcrumbs, and Localize-and-Stitch if implementation allows.

Data-dependent but label-free baselines.

RegMean, AdaMerging, APL, and any FusionBench-compatible unsupervised or activation-based method that can use the same number of unlabeled examples.

Labeled-replay baselines.

Ordinary replay-buffer gradient descent from the same task-arithmetic initialization; distance-scaled gradient descent without the attribution gate; head-only replay calibration; encoder-only replay fine-tuning; encoder-plus-head replay fine-tuning; multitask replay fine-tuning from the pretrained checkpoint; and multitask replay fine-tuning from the task-arithmetic merge.

Sanity controls.

Inverted gate, random gate with matched density, wrong-expert directions, shuffled replay labels, frozen first-step gates, and the aggregated- U version of the method. The aggregated- U ablation is important because the proposed algorithm applies each task update immediately after it is computed.

All labeled-replay baselines should use the same replay examples, the same number of gradient evaluations where possible, and the same hyperparameter search budget. This is essential because the strongest alternative explanation for a positive result is simply that the method performs supervised post-merge replay fine-tuning.

Metrics. For each GLUE task, we report the standard validation metric: Matthews correlation for CoLA, accuracy for SST-2, matched/mismatched accuracy for MNLI, correlation for STS-B, and accuracy/F1-style scores for the paraphrase and question-pair tasks according to the GLUE convention. Raw task scores, expert scores, pretrained scores, task-arithmetic scores, and absolute drops from the corresponding expert are primary. Because correlation-style metrics can be near zero or negative, the normalized retention metric is

$$\text{NormRet}_t = \frac{\text{Score}_t(\theta_{\text{merge}}) - \text{Score}_t(\theta_0)}{\text{Score}_t(\theta_t) - \text{Score}_t(\theta_0) + \epsilon_{\text{metric}}}, \quad (21)$$

where ϵ_{metric} prevents division by very small denominators. We report mean normalized retention, worst-task normalized retention, harmonic mean normalized retention, and expert-relative drop. Worst-task retention is important because a merge can improve average score while catastrophically harming a small task such as RTE or CoLA. For each main table, we also report confidence intervals across expert fine-tuning seeds and replay-buffer seeds.

Task-similarity and interference analysis. We group GLUE-8 tasks into interpretable clusters. The paraphrase and similarity cluster contains {MRPC, QQP, STS-B}; the entailment-style cluster contains {MNLI, QNLI, RTE}; and the single-sentence cluster contains {CoLA, SST-2}. We test whether expert-direction-gated refinement helps most in related task clusters, where movement toward one expert is more likely to be compatible with other tasks. We also compute task-vector cosine similarity, cross-task transfer performance, average attribution-patching sign agreement, and the cross-task interference matrix

$$C_{j \leftarrow i}^{(s,k)} = \left\langle \nabla_{\theta} \mathcal{L}_j(\theta^{(s,k-1)}; \mathcal{D}_j^{\text{probe}}), u_i^{(s,k)} \right\rangle. \quad (22)$$

Negative entries indicate that task i 's update is locally predicted to help task j , while positive entries indicate predicted interference. This matrix directly tests whether the gate is only self-protective or also tends to align across related tasks.

3.1 Ablations and Diagnostics

We plan the following ablations:

1. **Gate sign and necessity:** compare the proposed gate $g_{i,r}v_{i,r} < 0$ with no gate, an inverted gate $g_{i,r}v_{i,r} > 0$, and a random mask of the same density.
2. **Gate confidence and granularity:** vary ϵ_{gate} , compare coordinate-wise gates with tensor-wise or block-wise gates, evaluate top- k most negative attribution masks, and test sign-stability masks formed from multiple replay minibatches.
3. **Sequential update rule:** compare the proposed immediate single-task update with the aggregated- U variant, fixed task order, cyclic task order, and random task order.
4. **Distance scaling:** compare ordinary replay-gradient descent, distance-scaled gradient descent without the gate, direct expert-interpolation steps $u_i \propto m_i v_i$, and the full gated update.
5. **Clipping and normalization:** compare no normalization, tensor-wise RMS normalization, per-task global norm clipping, and post-update validation line search.
6. **Replay budget and variance:** vary the number of replay examples per task and report variation across replay samples and random seeds.
7. **Number of refinement steps:** compare $S \in \{0, 1, 2, 5, 10\}$ to test whether most gains occur in the first few steps or whether overfitting appears.
8. **Expansion point:** recompute gradients and gates before every single-task update versus freezing the first-sweep gates from $\theta^{(0)}$.
9. **Label and expert sanity checks:** compare correct labels with shuffled replay labels, and correct expert directions with wrong-expert directions.

3.2 Hypotheses

The project evaluates five hypotheses:

H1: Attribution-patching signs predict useful expert directions.

Coordinates or blocks with $g_{i,r}v_{i,r} < 0$ should be more likely to reduce task i 's loss when moved toward expert i than coordinates or blocks with $g_{i,r}v_{i,r} > 0$.

H2: Expert-guided replay refinement improves task arithmetic.

Starting from the same task-arithmetic initialization and using the same replay budget, the gated update should outperform ordinary replay-buffer gradient descent and distance-scaled gradient descent without the gate.

H3: Sequential single-task updates improve worst-task retention.

Applying each clipped task update immediately after it is computed should reduce overshoot and stale-gradient effects relative to an aggregated- U update, improving worst-task retention under the same replay budget.

H4: Benefits are larger for related tasks.

The method should help most in related task clusters, especially MRPC, QQP, and STS-B, where expert directions are more likely to be mutually compatible.

Table 1: Three-task RoBERTa-base proof of concept (MRPC, STS-B, SST-2; $n = 64$, $S = 5$). Mean normalized retention; best learning rate per family. Left: best configuration, mean $\pm$ std over five seeds. Right: a single-seed contrast at a large fixed learning rate, isolating the gate’s effect.

Method	Best mean NormRet ($\pm$ std)	at $\eta = 64$ (1 seed)
Task-arithmetic merge	0.558	—
Ordinary replay GD	0.831 ± 0.005	—
Ungated distance-scaled (no gate)	0.835 ± 0.017	-1.07 (diverges)
APR (gated, proposed)	0.840 ± 0.021	0.849 (stable)

H5: Gains are not GLUE-specific.

A positive GLUE result should transfer at least partially to a shared-head text-to-text setting or to the CLIP/ViT vision task-vector suite.

4 Preliminary Results

We report initial experiments in the RoBERTa-base multi-head setting (oracle task identity at evaluation). Unless noted, refinement uses S sweeps from the task-arithmetic merge with the corrected trust region applied *after* the learning rate (Eq. (11)) and threshold $\epsilon_{\text{gate}} = 0$. Each method family (ordinary replay gradient descent, the proposed gated update, and the ungated distance-scaled update) is given its own learning-rate search of equal size; we report the best configuration per family by mean normalized retention (Eq. (21)). These results are preliminary: the three-task study uses five seeds, but the GLUE-8 study is currently single-seed, $\lambda_i = 0.3$ is untuned, and the multi-head setting assumes oracle task identity.

Three-task proof of concept. On the MRPC/STS-B/SST-2 triple ($n = 64$ replay examples per task, $S = 5$), all refinement methods recover most of the gap from the task-arithmetic merge (mean normalized retention 0.558) toward the experts, and the three families are statistically tied at their tuned optima (Table 1). The decisive difference is robustness to the learning rate: the gated update attains its best score at a much larger step ($\eta = 64$) and remains stable there, whereas the ungated update—identical except for the gate—*diverges* at the same learning rate (mean normalized retention -1.07). The attribution gate thus acts as a protective mask that enables large, safe steps by suppressing the locally loss-increasing expert directions.

GLUE-8 scale-up. Merging all eight experts (one consistent RoBERTa-base fine-tuning source) is a much harder, more interference-prone setting: the task-arithmetic merge attains only mean normalized retention 0.320, with MRPC driven below the pretrained baseline. Here the value of expert anchoring grows: both anchored updates clearly outperform ordinary replay gradient descent, by a larger margin than in the three-task case and especially on worst-task retention (Table 2). The gate is approximately performance-neutral relative to the ungated anchored update on mean retention—the ordering flips between the two budgets and the gaps are within single-seed noise—but it again provides a wider safe learning-rate range (the gated update peaks at $\eta = 32$ and degrades gracefully, while the ungated update peaks at $\eta = 2-4$ and diverges by $\eta \gtrsim 8$). MRPC and CoLA remain the persistent worst tasks; QQP, STS-B, MNLI, and QNLI recover to 0.78–0.86.

Table 2: GLUE-8 (all eight tasks, RoBERTa-base, single seed). Best mean normalized retention per family (worst-task retention in parentheses); the task-arithmetic merge baseline is 0.320 (worst -0.175).

Method	$n=64, S=5$	$n=32, S=10$
Ordinary replay GD	0.574 (0.04)	0.588 (0.04)
Ungated distance-scaled (no gate)	0.637 (0.15)	0.607 (0.05)
APR (gated, proposed)	0.614 (0.14)	0.617 (0.05)

What the attribution gate is and is not doing. Two diagnostics temper the interpretation of the gate. First, the gate keeps a strikingly stable $\approx 36\%$ of coordinates across sweeps, tasks, and learning rates. A control attributes most of this to gradient sparsity rather than to the expert geometry: roughly 31% of encoder coordinates are word-embedding rows for tokens absent from the small replay buffer and therefore have exactly zero gradient (always masked), and a random direction of matched magnitudes reproduces almost the same density (0.345 vs. 0.361 for the true expert direction). The expert direction itself contributes only a small signal beyond chance. Second, the trust-region clip is almost never active at useful learning rates ($< 0.1\%$ of gated coordinates), functioning as a rare-event safeguard against the few large-gradient coordinates rather than as a routine operation. The gate’s measurable benefit is therefore concentrated in learning-rate robustness—masking the loss-increasing active coordinates so that large steps remain safe—rather than in a higher peak score at a tuned learning rate.

CLIP/ViT vision track. We next test whether the method transfers beyond GLUE encoders to the standard eight-task CLIP task-vector suite (SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD) with a CLIP ViT-B/32 backbone. Here the mergeable model is the image encoder, and each task keeps a *frozen* zero-shot classification head derived from the CLIP text encoder over class-name prompts; the experts are publicly available fine-tuned image encoders, merged with uniform $\lambda_i = 0.3$. Refinement uses $n = 64$, $S = 5$, and the trust region of Eq. (11) with $\gamma = 1$ (single seed). This is a stronger test than GLUE: a different modality, dense gradients without the word-embedding sparsity discussed above, and larger domain shifts. The task-arithmetic merge is weak in this regime (mean normalized retention 0.270, worst-task -0.527 ; SUN397 and Cars are pushed *below* the zero-shot backbone). All three families recover substantial ground, but—in contrast to GLUE-8—the attribution gate now attains both the best mean retention and the only positive worst-task retention (Table 3). The gated update improves the merge from 0.270 to 0.598 mean and from -0.527 to $+0.159$ worst-task, beating ordinary replay gradient descent by $+0.14$ mean / $+0.36$ worst and the ungated anchored update by $+0.06$ mean / $+0.25$ worst. The gains concentrate on the most interference-damaged tasks: Cars and SUN397, which the merge drove below the zero-shot floor, are precisely the ones the gate rescues (Table 4), whereas the ungated update sacrifices Cars catastrophically at its larger steps (worst-task -2.97 at $\eta = 8$). An inverted-gate control diverges (mean -1.68), confirming that the sign of the attribution product carries the signal. A smaller three-task vision study (EuroSAT/GTSRB/MNIST, single seed) shows the same ordering more sharply: APR 0.940 mean / 0.915 worst, versus 0.917/0.876 for the ungated update, 0.874/0.792 for ordinary replay GD, and 0.811/0.721 for the merge. As in GLUE, the best learning rate falls as tasks are added (the gated update peaks at $\eta = 32$ on three tasks and at $\eta = 8$ on eight). Together these results support H5—the method transfers to a second modality—and indicate that the gate’s usefulness is *modality-dependent*: approximately neutral on GLUE encoders

Table 3: Eight-task CLIP ViT-B/32 suite (single seed, $n = 64$, $S = 5$, $\gamma = 1$, uniform $\lambda_i = 0.3$). Best learning rate per family by mean normalized retention (Eq. (21)); worst-task retention in parentheses. The gated update is best on both, and is the only method keeping worst-task retention positive.

Method	Mean NormRet	Worst-task
Task-arithmetic merge	0.270	−0.527
Ordinary replay GD	0.458	−0.203
Ungated distance-scaled (no gate)	0.536	−0.086
APR (gated, proposed)	0.598	+0.159
Inverted-gate control	−1.682	−5.386

Table 4: Per-task top-1 accuracy on the eight-task CLIP suite: zero-shot backbone, task expert, task-arithmetic merge, and the proposed gated refinement (APR, $\eta = 8$). Refinement recovers most of the merge’s interference loss, including on Cars and SUN397, which the merge had degraded below the zero-shot backbone.

	SUN397	Cars	RESISC	EuroSAT	SVHN	GTSRB	MNIST	DTD	Avg.
Zero-shot	0.632	0.597	0.582	0.450	0.315	0.325	0.482	0.439	0.478
Expert	0.749	0.783	0.949	0.991	0.972	0.989	0.996	0.794	0.903
Merge	0.570	0.553	0.628	0.734	0.778	0.685	0.961	0.472	0.673
APR	0.650	0.648	0.769	0.923	0.874	0.851	0.961	0.578	0.782

but beneficial, on both mean and worst-task retention, on CLIP/ViT.

Replay budget and refinement horizon (CLIP/ViT). Two further studies on the eight-task suite probe the labeled-replay budget and the number of refinement sweeps, comparing the gated update directly against ordinary replay gradient descent—the strongest baseline that also uses labeled replay. First, reducing the replay buffer from $n = 64$ to 32 and 16 examples per task degrades both methods gracefully, but the gated update retains an almost constant advantage of $\approx +0.10$ mean normalized retention at every budget and remains the only method with non-negative worst-task retention down to $n = 16$ (Table 5). Its best learning rate also decreases as the buffer shrinks (from $\eta = 8$ at $n = 64$ to $\eta = 6$ at $n \leq 32$), consistent with noisier replay gradients favoring smaller trust-region steps. Second, on the refinement horizon: the gated update is essentially converged after five sweeps (mean retention $0.598 \rightarrow 0.615$ from $S = 5$ to $S = 15$ at $n = 64$, with no sign of overfitting the buffer), whereas ordinary gradient descent is still improving at $S = 35$ ($0.458 \rightarrow 0.504 \rightarrow 0.526$ at $S = 5, 15, 35$) yet does not catch up and—critically—never lifts its worst task (SUN397) back above the zero-shot backbone. Thus the expert anchor, rather than additional replay data or gradient steps, is what recovers the most interference-damaged tasks.

5 Expected Contribution

The expected contribution is an attribution-patching perspective on replay-guided model merging. Rather than using attribution scores to build a pre-merge sparsification mask, the method starts from a standard task-arithmetic merge and asks a local intervention question: for task i , which coordinates would reduce the replay loss if the current model moved toward expert i ? The answer is the sign of the coordinate-wise product $g_{i,r}v_{i,r}$, which coincides with a trust-region acceptance rule that

Table 5: Replay-budget study on the eight-task CLIP suite (mean normalized retention; worst-task in parentheses). Best configuration per family; the gated update (APR) near-converges by $S = 5\text{--}10$ sweeps, while ordinary replay GD is given a longer horizon ($S = 25\text{--}35$) as a generous baseline. APR holds an $\approx +0.10$ mean advantage at every budget and is the only method with non-negative worst-task retention down to $n = 16$.

Replay budget	APR (ours)	Ordinary replay GD
$n = 16$	0.522 (+ 0.018)	0.427 (−0.266)
$n = 32$	0.570 (+ 0.122)	0.476 (−0.197)
$n = 64$	0.598 (+ 0.159)	0.526 (−0.135)

keeps each step moving toward its expert, so the attribution-patching and replay-refinement views motivate the same procedure. This leads to a simple implementable algorithm: mask out expert directions that are locally loss-increasing, scale the remaining negative-gradient update by distance to the expert, clip each per-task update by a trust-region fraction, and immediately apply that task’s update before computing the next task’s gradient and gate. The proposal also contributes a stricter empirical protocol: fair labeled-replay baselines, an aggregated-update ablation, mask-stability tests, systematic GLUE pair/subset evaluations, and secondary shared-head or CLIP/ViT benchmarks. If successful, the method would provide a lightweight bridge between checkpoint-only task arithmetic and full multitask fine-tuning with replay buffers, while making clear which gains come from the attribution gate rather than from supervised replay alone.

References

- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.
- Michael S. Matena and Colin A. Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022.
- Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *International Conference on Learning Representations*, 2023.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.
- Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.
- Mohammad Davari and Eugene Belilovsky. Model Breadcrumbs: Scaling multi-task model merging with sparse masks. *arXiv preprint arXiv:2312.06795*, 2023.
- Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2024.
- Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-Merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.
- Fanshuang Kong, Richong Zhang, and Ziqiao Wang. Activated parameter locating via causal intervention for model merging. *arXiv preprint arXiv:2408.09485*, 2024.
- Shwai He, Runqi Wang, Jindong Wang, Jindong Gu, and Xing Xie. Localize-and-Stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.
- Zhenyi Lu, Chengxu Yin, Wujie Wang, and Xuebo Liu. Twin-Merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.
- Anke Tang, Li Shen, Yong Luo, Liang Ding, Han Hu, Bo Du, and Dacheng Tao. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.
- Zihan Zeng, Jiahui Chen, Lei Li, and Min Zhang. Efficient model editing with task vector bases: A theoretical framework and scalable approach. *arXiv preprint arXiv:2502.01015*, 2025.
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning*, 2017.
- Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2024.

Janos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing language model behavior to components. *arXiv preprint arXiv:2403.00745*, 2024.